

Agentic AI Mentorship: An 8 Week Programme

Agentic AI Mentorship: An 8 Week Programme

From foundations to shipping production agents

Overview

Duration	8 weeks
Schedule	2 sessions per week × 2 hours = 4 hours of direct teaching weekly
Format	Live working sessions. We code together, look at real production code where it helps, and you build between sessions.
Stack	OpenAI Agents SDK · FastAPI · PostgreSQL with pgvector · Claude Code (dev environment) · Supabase
Outcome	7 to 8 portfolio ready agent projects, working production fluency in a modern agentic stack, and the technical depth to design and ship agentic systems.

Working Principles

- **Project based throughout.** Every week ends with a small ship. By Week 8 you've built 7 to 8 agents of increasing complexity.
- **Case studies when they help, not on a schedule.** Some weeks we go deep on a real production project to see the wiring. Other weeks we just build. I'll draw on the case study library at the end of this doc whenever it saves us time or sharpens a concept. Not every week needs one.
- **Python is essential.** You don't need to learn it from scratch, but you'll write it daily. Async, types, Pydantic, decorators. All woven into the actual code we write together. By Week 3 you'll read and debug Python at speed. AI writes code, you verify it. Same principle as not trusting generated Terraform blindly.
- **Claude Code from day 1.** You learn it by using it. Not a separate module. It's your dev environment from session one.
- **Agents are services, not scripts.** From Week 2 onward, every agent we build is exposed via FastAPI. Real world agents live behind an API, so we start that way.
- **Your DevOps background is leverage, not baggage.** We skip roughly 80% of what most agentic AI courses spend time on (Docker, CI/CD, cloud basics) and use that time to go deeper on the AI layer.

Module Breakdown

Week 1 · Foundations and Mental Models

Topics - What an LLM actually is. Tokens, context, completions, why it generates what it generates. - Prompt engineering. System, user, and assistant messages. Few shot. Structured outputs. - Context engineering as the central skill of agentic work. - Claude Code setup as your daily dev environment. - *Python woven in*: async and await mental model, type hints, Pydantic basics.

Mini Project A structured output classifier or extractor using the OpenAI API directly. No agents yet. You'll feel the difference between "prompting" and "engineering context."

Week 2 · Your First Real Agent

Topics - What an agent actually is. The loop, the orchestrator, why "agent" is a fuzzy word. - OpenAI Agents SDK from zero. Agent, Tool, Runner, basic patterns. - Tool design. Function signatures, descriptions, when to split vs combine tools. - Exposing the agent as a FastAPI service. Routes, async endpoints, Pydantic request and response models. - Error handling and tool failure modes. - *Python woven in*: function signatures, decorators, Pydantic for tool args, FastAPI basics.

Mini Project A single agent assistant exposed as a FastAPI service with 3 to 4 tools. For example, a personal task tracker behind a `/chat` endpoint with create, list, update, plus calendar lookup, plus web search.

Week 3 · Memory, State, and Conversations

Topics - Conversation history management. What to keep, what to drop, what to summarise. - Context window pressure and summarisation strategies. - Short term vs long term memory as concepts and patterns. - Persisting state to Postgres from a FastAPI service. - *Python woven in*: async DB patterns with SQLAlchemy and asyncpg, FastAPI dependency injection.

Mini Project Add persistent memory to Week 2's assistant. It survives across sessions, remembers user preferences, and summarises old conversation history.

Week 4 · RAG (Retrieval Augmented Generation)

Topics - Why RAG exists, what problem it actually solves, when it's the wrong answer. - Vector embeddings. The mental model, not just the API. - Postgres plus pgvector setup from scratch. - Embedding pipelines and chunking strategies. - Retrieval strategies. Semantic, hybrid (BM25 plus vector), metadata filtering. - A pass over reranking and retrieval evaluation. We touch it, not master it. - *Python woven in*: working with numpy style vectors, batch processing.

Mini Project A RAG powered domain expert. For example, a legal docs assistant or a product knowledge bot, with hybrid retrieval.

Week 5 · Multi Agent Orchestration

Topics - When you actually need multiple agents (and when one good agent is enough). - The planning problem. - Orchestrator pattern, deep. - Agent as a tool pattern, deep. Why it's a better default than handoffs in most production cases. - A pass over handoffs and routing. We touch it, not master it. - Sub agent context isolation.

Mini Project A multi agent system with an orchestrator plus 2 to 3 specialist agents. For example, a research then draft then review pipeline.

Week 6 · MCP (Model Context Protocol)

Topics - What MCP is, why it exists, where it fits in the stack. - Consuming existing MCP servers from inside your agents. - Building your own MCP server from scratch. - MCP vs custom tool definitions. When to choose which.

Mini Project Build an MCP server exposing a custom domain API and connect it to Claude Code or your own agent.

Week 7 · Production Hardening

Topics - Eval driven development. Why evals matter more than tests in agentic systems. - Building real eval suites for your agents. - Cost optimisation. Prompt caching, model routing, token budgets. - Latency optimisation. Streaming, parallel tool calls. - Hallucination mitigation. Guardrails, confidence scoring, structured outputs as a defence. - Observability. Logging, tracing, monitoring agent runs.

Mini Project Take an earlier agent and add a real eval suite, guardrails, and observability. Deploy to a staging environment.

Week 8 · Deployment and Multitenancy

Topics - Deployment for agentic systems (your DevOps background carries this module, we move fast). - Background tasks for long jobs (ARQ plus Redis). - Multitenancy basics. Org scoped data, JWT auth (JWKS), per tenant rate limiting. - Webhook handling for production integrations.

Mini Project A multitenant deployment of an earlier agent with proper JWT auth and per tenant rate limits. The capstone of the programme.

Case Study Library

These are real production projects I've shipped. I'll draw on them across the course when a concept is easier to grasp by walking through real wired code than by explaining it abstractly. Not every week pulls one. Some weeks we just build.

- **Multitenant production WhatsApp AI agent.** Orchestrator pattern, conversation state, RAG, the full production stack. The richest reference for most of the second half of the course.
 - **Single tenant to multitenant evolution.** What breaks when you scale up the tenancy model, what architectural debt looks like, and how to refactor out of it.
 - **Autonomous lead generation agent.** Learning engine pattern, an `experiments` table, multi LLM fallback chains, philosophy of agents that improve over time.
 - **Agentic 7 stage content pipeline.** Multi LLM coordination (Claude, GPT 4o, Gemini), structured outputs at each stage, human review in the loop.
 - **Real time 4 agent crisis response pipeline.** Signal ingestion to detection to planning to execution, confidence based handoff, deterministic tool scoring.
 - **Internal staff facing agent with RBAC.** Agent as a tool pattern (orchestrator plus specialist sub agents), draft and confirm mutations, role based access enforced in code.
 - **Production security audit.** Twelve vulnerabilities, all tracing to a single root cause (no auth spine). What went wrong and how it was fixed. Powerful for the hardening modules.
-

What You'll Have at the End

- **7 to 8 working agent projects** of increasing complexity, all in your portfolio.
- **Production fluency** in the OpenAI Agents SDK plus FastAPI plus Postgres plus pgvector stack.
- **Working Python competence.** Read, write, and debug at speed. You'll know when AI generated code is wrong.
- **Claude Code as a daily tool.** You'll build faster than the majority of engineers I work with.
- **The vocabulary and mental models** to design agentic solutions for real problems with confidence.
- **A real production grade agent** running end to end in your portfolio.

What You'll Be Expected to Bring

- 4 hours per week of live time with me, plus realistically 6 to 10 hours per week of your own build time between sessions.
- A working laptop and a **Claude Pro subscription**. Claude Code is your daily dev environment, this is mandatory.
- LLM API access. We'll talk through OpenAI and Anthropic API key setup, billing, and budgets together in Week 1.

What's Out of Scope

- ML fundamentals or training models from scratch (we use models, we don't build them).
 - Heavy theory or research papers.
 - Deep front end engineering. When we need a UI to demo something, we'll have Claude Code build it. No deep dives on React patterns, CSS, or design systems.
 - Sales, pricing, or business coaching.
-

Outline prepared by Hashir · Lead AI Engineer · Adjust as we go.